

Phase 1-6. 멀티테넌시 (Global Query Filter) 학습정리

이 문서의 목적 — 1-F 멀티테넌시를 처음부터 끝까지 복습한다. 용어를 하나하나 풀어 설명하고, 남들이 많이 하는 실수를 정리했다. 특히 **두 부분은 특별히 상세하게** 다룬다 (작업 중 집중이 흐트러진 구간): ① Global Query Filter의 함정 (인증 흐름이 필터로 깨지는 문제 + IgnoreQueryFilters) ② 멀티테넌시 신원 설계 (이메일 전역 유일 vs 테넌트 내 유일) 이 두 개는 ★★★ 표시로 강조했다. 나머지는 늘 하던 형식.

0. 큰 그림 — 우리가 만든 것

여러 정비소(테넌트)가 하나의 예약 SaaS를 **공유**하되, 각자 데이터는 **격리**되게 만들었다. 강남정비소는 강남 데이터만, 부산정비소는 부산 데이터만 본다. 서로 절대 안 보인다.

핵심은 — 개발자가 매번 "내 테넌트 것만" 필터를 손으로 안 붙여도, EF Core가 **자동으로** 붙여주게 만든 것. 그래서 실수로 데이터가 새는 걸 원천 차단했다.

그리고 어제(1-E) JWT에 심어둔 `tenantId` 가 오늘 드디어 쓰였다. "매 요청의 신분증(JWT)에서 소속(tenantId)을 읽어 그 소속 데이터만 보여준다."

1. 멀티테넌시 기본 용어

용어: 테넌트 (tenant)

테넌트 = "우리 서비스를 쓰는 하나의 고객사(조직)". 우리 경우 정비소 하나가 테넌트. "입주사" 개념 — 한 건물(시스템)에 여러 입주사(정비소)가 세들어 산다.

용어: 멀티테넌시 (multi-tenancy)

멀티테넌시 = "하나의 시스템을 여러 테넌트가 공유하되, 데이터는 서로 격리되는 구조" - 공유 — 모든 정비소가 같은 서버·코드·DB를 씀 (각자 서버 안 만듦) - 격리 — 근데 각 정비소는 자기 데이터만 봐야 함

용어: TenantId

각 데이터 행이 "어느 테넌트 소속인지" 나타내는 식별자. 우리 엔티티는 `TenantEntity` 를 상속하면 `TenantId` 를 가진다 (User가 그렇다). 한 테이블에 여러 테넌트 데이터가 섞여 있고, 이 TenantId로 구분한다.

2. 왜 격리가 어려운가 — 필터를 까먹는 함정

우리는 모든 정비소 데이터를 **한 DB, 한 테이블**에 넣는다 (Shared DB 방식). 각 행에 TenantId가 있다.

Users 테이블:

Id	Email	TenantId
1	gangnam@test.com	강남정비소
2	busan@test.com	부산정비소

한 테이블에 섞임

문제: 매 쿼리마다 "내 테넌트 것만" 필터를 붙여야 한다.

맞게: `SELECT * FROM Users WHERE TenantId='강남'`
까먹으면: `SELECT * FROM Users` (부산 것까지 다 보임! 데이터 유출)

딱 한 번만 WHERE를 까먹어도 다른 정비소 데이터가 유출된다. 이게 멀티테넌시의 제일 무서운 함정. 실제 업계에서도 "A 회사 로그인했는데 B 회사 데이터가 보였다" 같은 사고가 종종 난다. 심각한 보안 사고다.

3. 해결 — EF Core Global Query Filter

용어: Global Query Filter (전역 쿼리 필터)

"모든 쿼리에 자동으로 특정 조건을 끼워 넣는 EF Core 기능"

한 번 설정하면 개발자가 `WHERE TenantId=...` 를 직접 안 써도 EF Core가 자동으로 붙인다.

```
개발자 코드:      m_Db.Users.ToListAsync()
EF Core 실제 실행: SELECT * FROM Users WHERE TenantId='현재테넌트'
                  ↑ 자동으로 붙임!
```

개발자는 필터를 **까먹을 수가 없다**. 코드에 안 써도 EF Core가 무조건 끼워주니까. 이게 1-F의 핵심 — "필터를 자동화해서 인간의 실수를 없앤다."

설정 코드

DbContext의 `OnModelCreating` 에서:

```
modelBuilder.Entity<User>()
    .HasQueryFilter(u => u.TenantId == m_TenantProvider.GetCurrentTenantId());
```

- `HasQueryFilter(...)` — "이 엔티티를 조회할 때마다 이 조건을 자동으로 붙여라"
- 이후 `m_Db.Users` 를 어디서 조회하든 `WHERE TenantId=현재테넌트` 가 자동으로 붙는다

왜 User에만 걸고 Tenant/RefreshToken엔 안 걸었나

- **User** — TenantEntity 상속(TenantId 있음). 테넌트 격리 대상 → 필터 O
- **Tenant** — 테넌트 자체는 격리 대상 아님(시스템 관리 영역) → 필터 X
- **RefreshToken** — BaseEntity 상속(TenantId 없음). 인증 인프라라 격리 대상 아님 → 필터 X

즉 **TenantId를 가진 엔티티(TenantEntity 상속)에만** 필터를 건다. Phase 2에서 Customer·Reservation이 생기면 그것들에도 건다.

4. 현재 테넌트는 어떻게 아나 — JWT의 tenantId (1-E와 연결)

Global Query Filter가 `WHERE TenantId=현재테넌트` 를 붙이려면, "지금 요청자가 어느 테넌트인지"를 알아야 한다. 그 정보가 **어제(1-E) JWT에 실어둔 tenantId**다.

```
어제(1-E): 로그인 → JWT 발급 (claims에 tenantId 넣음) ← "왜 넣지?" 했던 그거
오늘(1-F): 요청마다 그 JWT가 올 → tenantId를 꺼냄 → 필터에 사용
```

용어: HttpContext

"현재 처리 중인 HTTP 요청의 모든 정보를 담은 객체". 요청 헤더, 쿠키, 그리고 인증된 사용자 정보(JWT claims)가 다 여기 있다. JWT 검증 미들웨어(1-E)가 토큰을 검증하고 나면 claims를 `HttpContext.User` 에 담아둔다.

용어: IHttpContextAccessor

"아무 데서나 현재 HttpContext에 접근하게 해주는 도구" - 문제: HttpContext는 컨트롤러에선 바로 쓰는데, DbContext·서비스 같은 곳에선 직접 접근 안 됨 - 해결: IHttpContextAccessor를 주입받으면 어디서든 현재 요청의 HttpContext를 꺼낼 수 있음 - 이걸 등록이 필요:

```
builder.Services.AddHttpContextAccessor();
```

TenantProvider — 현재 테넌트를 꺼내는 장치

어제 배운 패턴(인터페이스는 Application, 구현은 Infrastructure)으로:

```
Application:      ITenantProvider (약속 - "현재 테넌트 Id를 알려줄 수 있다")
                  ↑ 구현
Infrastructure:   TenantProvider (HttpContext에서 tenantId 꺼내는 실제 구현)
```

구현의 핵심 (TenantProvider.GetCurrentTenantId):

```
var user = m_HttpContextAccessor.HttpContext?.User;    // 현재 요청의 사용자
if (user is null) return null;

var tenantIdClaim = user.FindFirst("tenantId");          // tenantId claim 찾기
if (tenantIdClaim is null) return null;                  // (1-E에서 넣은 그 이름)

if (Guid.TryParse(tenantIdClaim.Value, out var tenantId)) // 문자열 → Guid 변환
    return tenantId;
return null;
```

- `FindFirst("tenantId")` — 1-E에서 `new Claim("tenantId", ...)` 로 넣은 걸 다시 찾음. 넣은 이름과 찾는 이름이 정확히 같아야 함 (둘 다 "tenantId")
- claim 값은 문자열이라 `Guid.TryParse` 로 다시 Guid 변환. TryParse는 실패해도 예외 안 나고 false 반환(안전한 변환)
- 인증 안 된 요청(로그인 전)은 null 반환 → 그래서 반환 타입이 `Guid?` (nullable)

FrameworkReference 함정

Infrastructure는 클래스 라이브러리(웹 프로젝트 아님)라 `IHttpContextAccessor` (웹 타입)가 기본으로 없다. csproj에 추가해야 한다:

```
<ItemGroup>
  <FrameworkReference Include="Microsoft.AspNetCore.App" />
</ItemGroup>
```

- 별도 NuGet 설치가 아니라, 이미 설치된 .NET의 웹 관련 타입 전체를 참조만 하는 것

5. ★★★ Global Query Filter의 함정 — 인증 흐름이 깨진다 (집중해서!)

여기가 이번 챕터 최대 함정이다. 특히 상세하게 설명한다.

문제의 본질 — 답이 먼저냐 달걀이 먼저냐

User에 Query Filter를 걸면, 모든 User 조회에 `WHERE TenantId=현재테넌트` 가 자동으로 붙는다. 좋은 일 같지만 — 로그인에 깨진다. 왜?
로그인 흐름을 따라가 보자:

1. 로그인 요청 도착 (이메일 `busan@test.com`, 비밀번호 xxx)
❌ 아직 로그인 전이다! JWT가 없다!
2. `AuthService.LoginAsync` 실행
3. 이메일로 User를 찾아야 함: `m_UserRepository.FindByEmailAsync("busan@test.com")`
4. 근데 Query Filter가 자동으로 붙는다:
`SELECT * FROM Users WHERE Email=busan@test.com AND TenantId=현재테넌트`
5. 근데 "현재테넌트"가 뭐지?
❌ `GetCurrentTenantId()`를 부르는데, 아직 로그인 전이라 JWT가 없다
❌ 그래서 null을 반환한다
6. 결국: `WHERE TenantId = null`
❌ NULL과 같은 TenantId는 아무것도 없다 (SQL에서 = NULL은 항상 거짓)
❌ User를 못 찾는다!
7. "이메일 또는 비밀번호가 올바르지 않습니다" ❌ 로그인 실패!!

왜 이게 모순인가 (핵심 이해)

로그인하려면 User를 찾아야 하는데, User를 찾으려면 필터가 "현재 테넌트"를 요구하고, 현재 테넌트를 알려면 이미 로그인이 돼 있어야 한다.

= 답이 먼저냐 달걀이 먼저냐 문제다. 순환에 빠진다.

이걸 그림으로:

```
로그인 시도 → User 조회 필요 → 필터가 테넌트 요구 → 테넌트는 로그인 후에 결정됨
      ↑                                     |
      └──────────────────────────────────┘
                    로그인 안 됨
```

방금 만든 인증(1-E)이 멀티테넌시(1-F) 때문에 깨지는 것이다.

해결 — IgnoreQueryFilters (필터 우회)

용어: IgnoreQueryFilters() "이 조회에서는 Global Query Filter를 무시해라"

인증 관련 조회(로그인, 회원가입 중복확인)는 **테넌트 격리 이전의 작업**이다. 그러니 이 조회만 필터를 우회해야 한다.

```
public async Task<User?> FindByEmailAsync(string email, CancellationToken ct = default)
{
    var normalized = email.Trim().ToLowerInvariant();
    return await m_Db.Users
        .IgnoreQueryFilters() // ← 필터 우회!
        .FirstOrDefaultAsync(u => u.Email == normalized, ct);
}
```

- `.IgnoreQueryFilters()` — 이 조회에선 `WHERE TenantId=...` 를 안 붙인다. 그래서 tenantId가 없어도(로그인 전이라도) 이메일로 User를 찾을 수 있다.

"필터를 우회하면 격리가 깨지는 거 아닌가?" — 아니다, 안전하다 (중요!)

당연히 드는 의문이다. "격리하려고 필터 걸었는데, 우회하면 격리가 무너지는 것 아냐?"

안전하다. 이유:

1. **로그인/회원가입은 "인증 전" 단계다.** 애초에 테넌트 격리가 적용될 수 없는 영역이다. "이 이메일의 유저가 누구인지" 찾는 건 테넌트를 가리기 전의 일이니까.
2. **로그인이 성공하면** → JWT에 그 User의 tenantId가 담기고 → **그 이후의 모든 요청은 필터가 정상 작동한다.**
3. 즉 구조가 이렇다: **"인증(입구)만 필터를 우회하고, 인증 후의 실제 데이터 접근은 다 필터가 지킨다."**

비유로 이해 (핵심)

건물 입구(로그인): "당신이 어느 회사 소속인지" 확인하는 단계
→ 아직 회사별 출입 제한(필터)을 적용 안 함
→ 신원 확인이 목적이니까. 입구에서까지 필터 걸면 아무도 못 들어옴 (= 로그인 깨짐)

신원 확인 끝(로그인 성공) → 그때부터 "당신 회사 층만 출입 가능"(필터) 작동
→ 이후 모든 데이터 접근은 필터가 지킴

입구에서까지 회사별 출입증 검사를 하면, 신원 확인 자체를 못 해서 아무도 못 들어온다. 그래서 입구(인증)는 필터를 우회하고, 들어온 다음(인증 후)부터 필터가 작동한다.

어느 메서드에 IgnoreQueryFilters를 쓰나

- `FindByEmailAsync` (로그인, 회원가입 중복확인) → 우회 O (인증 전 작업)
- `FindByIdAsync` (갱신 시 유저 찾기) → 우회 O (Refresh 토큰으로 유저 찾는 인증 작업)
- `GetAllAsync` (일반 목록 조회) → 우회 X! 필터가 작동해야 함 (격리 대상)
- `AddAsync` (저장) → 조회가 아니라 해당 없음

핵심 대비: 인증용 조회는 필터 우회, 일반 조회는 필터 적용. "인증은 격리 이전, 일반 조회는 격리 이후"라는 구분이 코드로 나타난다.

이게 왜 강력한 면접 포인트인가

"Global Query Filter를 걸면 인증 흐름(로그인)이 깨집니다. 로그인엔 테넌트가 정해지기 전 단계인데 필터는 테넌트를 요구하기 때문입니다. 그래서 인증 관련 조회는 IgnoreQueryFilters로 우회하고, 인증 이후의 데이터 접근은 모두 필터가 지키게 했습니다." → 이걸 이해하고 설명하면 "멀티테넌시를 말이 아니라 진짜 구현해봤다"는 강한 시그널. 이 함정을 안 겪은 사람은 이 답을 못 한다.

6. ★★★ 멀티테넌시 신원 설계 — 이메일 전역 유일 vs 테넌트 내 유일 (집중해서!)

여기도 특별히 상세하게. 설계 선택의 문제라 이해가 중요하다.

문제 제기

로그인할 때 우리는 이렇게 한다:

```
FindByEmailAsync(email) // IgnoreQueryFilters - 테넌트 무시하고 이메일로만 찾을
```

테넌트 구분 없이 **이메일만으로** User를 찾는다. 이 말은 — **이메일이 전체에서 유일해야** 이게 제대로 작동한다는 뜻이다.

만약 강남정비소에 **test@test.com**, 부산정비소에도 **test@test.com** 이 있으면?

로그인: test@test.com 으로 찾기 (테넌트 무시)

❗ 두 명이 나온다! ❗ 누구지??

❗ FirstOrDefault라 아무나 하나 걸릴 ❗ 엉뚱한 사람으로 로그인될 수 있음 (버그/보안사고)

그래서 지금 우리 구조는 "이메일은 테넌트가 달라도 중복 불가(전역 유일)"를 전제한다.

근데 이게 유일한 방법은 아니다 — 두 가지 설계 방식

멀티테넌시에서 "사용자 신원"을 잡는 방식은 두 가지다. 이건 필수가 아니라 **설계 선택**이다.

방식 1 — 이메일 전역 유일 (우리가 택한 것)

"이메일 하나 = 계정 하나. 테넌트 무관하게 유일."

test@test.com ❗ 무조건 한 명 (강남이든 부산이든 한 곳에만 존재 가능)

- **로그인:** 이메일 + 비밀번호. 테넌트 안 물어봄 (이메일이 유일하니 바로 특정됨)
- **장점:** 로그인만 단순 (사용자가 "어느 정비소" 안 골라도 됨)
- **단점:** 같은 사람이 여러 정비소에 소속될 수 없음 (kim@gmail.com이 강남 직원이면서 부산 직원? → 불가능)
- **실제 예시:** 많은 B2B SaaS의 시작 방식 (이메일 = 글로벌 계정)

방식 2 — 테넌트 내에서만 유일

"이메일은 테넌트 안에서만 유일. 테넌트가 다르면 같은 이메일 OK."

강남정비소: test@test.com ❗

부산정비소: test@test.com ❗ 둘 다 존재 가능 (서로 다른 계정)

- **로그인:** 이메일 + 비밀번호 + **테넌트 지정**이 필요 ("어느 정비소로 로그인?"을 알아야 누구인지 특정됨)
- 보통 **서브도메인**으로 해결: gangnam.myapp.com, busan.myapp.com → URL이 테넌트를 알려줌
- 또는 로그인 화면에서 회사 코드 입력
- **장점:** 같은 사람이 여러 테넌트에 소속 가능
- **단점:** 로그인이 복잡 (테넌트를 먼저 알아야 함)
- **실제 예시:** Slack 현재 방식 (워크스페이스별 로그인), 많은 엔터프라이즈 SaaS

비교표

	방식 1 (전역 유일)	방식 2 (테넌트 내 유일)
이메일 유일성	전체에서 유일	테넌트 안에서만 유일
로그인 입력	이메일 + 비밀번호	이메일 + 비밀번호 + 테넌트(서브도메인 등)
같은 사람 다중 소속	불가	가능
로그인 복잡도	단순	복잡
우리 선택	이걸 택함	

우리가 방식 1을 택한 이유

- **단순함** — 로그인이 이메일+비밀번호로 깔끔. 서브도메인 라우팅 같은 복잡도 없음
- **스코프 관리** — 방식 2는 서브도메인, 테넌트별 로그인 화면 등 결가지가 많음. 포트폴리오 스코프엔 과함
- **대부분 SaaS 기본** — 많은 서비스가 이메일=글로벌 계정으로 시작

방식 1을 "제대로" 하려면 — 이메일 유니크 제약 (놓쳤다가 발견)

방식 1(전역 유일)을 택했으면, DB에서 "이메일 중복 저장 자체를 막아야" 경고하다. 처음엔 이게 빠져 있었다 (AuthService 코드로만 중복 확인). 그래서 추가했다:

```
// DbContext.OnModelCreating
modelBuilder.Entity<User>()
    .HasIndex(u => u.Email)
    .IsUnique();
```

- 같은 이메일을 두 번 저장하려 하면 **DB가 거부**한다
- AuthService의 중복 확인 코드가 실수로 빠져도 **DB가 최후의 방어선**이 됨
- 이걸 스키마 변경이라 마이그레이션 필요 (AddUserEmailUniqueIndex)

이중 방어: 애플리케이션(AuthService 중복 확인) + DB(유니크 제약). 어느 하나가 뚫려도 다른 하나가 막는다. 견고한 설계의 기본.

이게 왜 좋은 면접 소재인가

"멀티테넌트에서 사용자 신원을 어떻게 잡았나요?" → "이메일 전역 유일 방식을 택했고, 그래서 로그인이 이메일+비밀번호로 단순합니다. 만약 같은 사용자가 여러 테넌트에 소속돼야 한다면 테넌트 내 유일 + 서브도메인 방식으로 갔을 겁니다. 트레이드오프는 로그인 단순성 vs 다중 소속 지원입니다. 그리고 전역 유일을 DB 유니크 제약으로 강제해서, 애플리케이션과 DB 양쪽에서 이중으로 보장했습니다." → 설계 선택의 이유 + 트레이드오프 + 이중 방어까지. 강력한 답변.

7. 격리 테스트 — 실제로 되는지 눈으로 확인

테스트용 조화 엔드포인트

격리를 눈으로 보려면 "보호된 조화 엔드포인트"가 필요했다. GET /api/v1/users를 만들었다:

```
[ApiController]
[Route("api/v1/users")]
[Authorize] // ← 로그인 필수
public class UsersController : ControllerBase
{
    [HttpGet]
    public async Task<IActionResult> GetAll(CancellationToken ct)
    {
        var users = await m_UserRepository.GetAllAsync(ct); // 필터 적용됨!
        var response = users.Select(u => new UserResponse(u.Id, u.Email, u.TenantId)).ToList();
        return Ok(response);
    }
}
```

- **[Authorize]** — 로그인해야만 접근 (1-E JWT 검증 미들웨어 작동). 토큰 없으면 401
- **GetAllAsync** 는 IgnoreQueryFilters를 **안** 씀 → Query Filter가 작동 → 현재 테넌트 것만 반환

용어: DTO로 민감 정보 제외

UserResponse(Guid Id, string Email, Guid TenantId) — PasswordHash를 **안** 담는다. User 엔티티엔 해시가 있지만, API 응답엔 Id/Email/TenantId만. 민감 정보(비번 해시)를 API로 내보내면 안 되니까. DTO로 "내보낼 것만 골라내는" 습관.

테스트 시나리오 (격리 증명)

1. 강남 유저로 로그인  Access Token 받기
2. 그 토큰으로 **GET** /api/v1/users  강남 유저만 나옴! (부산 안 보임)
3. 부산 유저로 로그인  다른 토큰
4. 그 토큰으로 **GET** /api/v1/users  부산 유저만 나옴!

같은 엔드포인트인데 토큰(=테넌트)에 따라 다른 데이터 = 격리 성공.

8. Scalar 자물쇠 (.NET 10 OpenAPI 최신 문법)

문제

[Authorize] 엔드포인트를 테스트하려면 매번 토큰을 헤더에 복사해야 해서 고통스러웠다. Scalar에 "토큰 입력란(자물쇠)"을 띄우려 했는데, .NET 10에서 OpenAPI API가 크게 바뀌어서 예전(.NET 9) 코드가 다 깨졌다.

용어: OpenAPI / 보안 스키마 / 보안 요구사항

- **OpenAPI** = API를 기술하는 표준 문서 (엔드포인트, 스키마, 인증 방식 등)
- **SecurityScheme(보안 스키마)** = "이 API는 이런 인증을 쓴다"는 정의 (예: Bearer JWT)
- **SecurityRequirement(보안 요구사항)** = "이 엔드포인트는 그 인증이 필요하다"는 적용
- 둘 다 있어야 UI에 자물쇠가 뜬다. 스키마만 정의하고 요구사항 적용 안 하면 안 뜬

.NET 10에서 바뀐 점 (핵심 함정)

OpenAPI.NET 2.x에서 API가 바뀌었다: - 구버전(.NET 9): `new OpenApiSecurityScheme { Reference = new OpenApiReference {...} }`
 - 신버전(.NET 10): `new OpenApiSecuritySchemeReference("Bearer", document) - document.SecurityRequirements` (문서 전역)
 → 없어짐 - 각 operation마다 `operation.Security.Add(...)` 로 적용 - `document.Components.SecuritySchemes[...] =`
`document.AddComponent(...)` - `Array.Empty<string>()` → `[]` (List 타입이라)

Document Transformer (별도 클래스)

BearerSecuritySchemeTransformer : IOpenApiDocumentTransformer 를 만들어서: - IAuthenticationSchemeProvider로 "Bearer 인증이 설정됐나" 확인 - 있으면 보안 스키마 정의(AddComponent) + 모든 operation에 요구사항 적용(foreach)

```
builder.Services.AddOpenApi(options =>
{
    options.AddDocumentTransformer<BearerSecuritySchemeTransformer>();
});
```

Scalar에 명시 (마지막 한 방)

문서는 완벽해졌는데 Scalar UI에 자물쇠가 안 떴다. 명시적으로 알려주니 떴다:

```
app.MapScalarApiReference(options =>
{
    options.AddPreferredSecuritySchemes("Bearer");
});
```

- Scalar가 "Bearer 인증을 기본으로 쓴다"고 인식 → 우측 상단에 토큰 입력란(자물쇠) 표시

사용법

로그인 → accessToken 복사 → Scalar 우측 상단 "Bearer Token" 칸에 붙여넣기 → 이후 모든 요청에 자동으로 Authorization: Bearer ... 붙음. 매번 헤더 복사 끝.

참고: Transformer가 모든 엔드포인트에 자물쇠를 걸어서 /health, login 등에도 "Auth Required"가 표시되지만, 이건 UI 표시일 뿐이다. 실제 인증 강제는 [Authorize]가 있는 곳만. UI 표시 ≠ 실제 강제.

9. 흔한 실수 총정리 (이번에 겪은 것들)

① IHttpContextAccessor 못 찾음

Infrastructure는 웹 프로젝트가 아니라 웹 타입이 없다. → csproj에 `<FrameworkReference Include="Microsoft.AspNetCore.App" />` 추가

② 로그인 필터 때문에 깨짐 (최대 함정)

Global Query Filter가 인증 조회에도 적용돼서 로그인 실패. → FindByEmailAsync/FindByIdAsync에 IgnoreQueryFilters() (5장 참고)

③ 이메일 유니크 제약 누락

방식 1(전역 유일)인데 DB 제약이 없어서, 이론상 중복 이메일이 저장될 수 있었음. → HasIndex(u => u.Email).IsUnique() + 마이그레이션 (6장 참고)

④ "참조 찾기"에 구현이 안 뜸

인터페이스 메서드의 "참조 찾기(Shift+F12)"는 호출처만 보여준다. 구현이 아니다. → 구현을 보려면 "구현으로 이동(Ctrl+F12)". VS의 세 이동 구분: F12=정의, Ctrl+F12=구현, Shift+F12=참조(호출처)

⑤ .NET 10 OpenAPI 문법 깨짐

.NET 9 예제 코드가 .NET 10에서 다 컴파일 에러. (SecurityRequirements 없음, Reference 없음, Array.Empty vs [] 등) → OpenApiSecuritySchemeReference, AddComponent, operation별 Security 적용 (8장 참고) → 교훈: 최신 버전은 API가 바뀔 수 있다. 버전 맞는 문서를 확인.

⑥ 401 = 정상 (오해 주의)

[Authorize] 엔드포인트를 토큰 없이 호출하면 401. 이건 에러가 아니라 "보호가 작동하는 것". 토큰을 넣으면 통과.

10. Silo vs Shared — 멀티테넌시 아키텍처 (면접 1번 질문)

두 가지 방식

Silo (사일로) — 테넌트마다 DB/인스턴스 분리

강남 → 강남 전용 DB
부산 → 부산 전용 DB (물리적으로 완전 분리)

- 장점: 완벽한 격리(섞일 수가 없음), 규제 대응 쉬움
- 단점: 비용·운영 부담 큼 (테넌트마다 DB 관리)

Shared (공유) — 한 DB + TenantId 구분 (우리가 택함)

강남·부산 → 하나의 DB, TenantId 컬럼으로 구분 + Global Query Filter

- 장점: 비용·운영 효율적
- 단점: 격리를 소프트웨어로 보장해야 함 (Query Filter의 역할)

★ 나의 강력한 카드 — 양쪽 다 경험

- 회사 IPlus = Silo (의료법상 기관별 데이터 분리 의무, 300개 클리닉 각 인스턴스)
- 03 예약 SaaS = Shared (한 DB + Global Query Filter)
- 면접: "Silo는 규제 도메인에서 격리가 완벽하지만 비용이 크고, Shared는 효율적이지만 소프트웨어로 격리를 보장해야 합니다. 도메인 특성(규제 강도, 비용, 격리 요구)에 따라 선택합니다. 저는 회사에서 Silo를, 03에서 Shared를 실제로 구현했습니다."
- "패턴을 아는 것"을 넘어 "실경험으로 트레이드오프를 설명" → 강력.

11. 이번에 만든/수정한 파일 지도

```
Reservation.Application/  
└─ Abstractions/ITenantProvider.cs      - 현재 테넌트 제공 약속  
  
Reservation.Infrastructure/  
├─ Tenancy/TenantProvider.cs            - HttpContext에서 tenantId 추출 (구현)  
├─ Persistence/ReservationDbContext.cs  - Query Filter + 이메일 유니크 (수정)  
└─ Repositories/UserRepository.cs      - IgnoreQueryFilters 추가 + GetAllAsync (수정)  
    (csproj에 FrameworkReference 추가)  
  
Reservation_API/  
├─ OpenApi/BearerSecuritySchemeTransformer.cs - Scalar 자물쇠용  
├─ DTOs/UserResponse.cs                  - 목록 응답 (PasswordHash 제외)  
├─ Controllers/UsersController.cs        - GET /users (격리 테스트용)  
└─ Program.cs                            - AddHttpContextAccessor, ITenantProvider DI,  
                                          OpenApi transformer, Scalar preferred scheme
```

12. 한 장 요약 (자기 전에 이것만)

1. 멀티테넌시 = 여러 테넌트가 시스템 공유하되 데이터는 격리
2. 합정 = 매 쿼리에 WHERE TenantId 까먹으면 유출
3. 해결 = Global Query Filter로 자동 적용 (까먹을 수 없게)
4. 현재 테넌트 = 1-E JWT의 tenantId를 TenantProvider가 추출
5. ★ 최대 합정 = 필터가 인증 흐름(로그인)을 깬다 (테넌트 없는 상태에서 User 조회) → IgnoreQueryFilters로 인증 조회만 우회. 인증 후 접근은 필터가 지킴
6. ★ 신원 설계 = 이메일 전역 유일(방식1, 우리 것) vs 테넌트 내 유일(방식2) → 방식1은 로그인 단순, 다중소속 불가. DB 유니크 제약으로 이중 방어
7. 격리 증명 = 토큰 따라 다른 데이터 (같은 엔드포인트, 다른 결과)

8. **Silo(회사) vs Shared(03)** = 양쪽 실경험이 면접 카드

작성: Phase 1-6 완료 시점. 다음 단계: 1-G 배포 검증 → v0.1 태그 (Phase 1 완료).